

SUMMER TRAINING/INTERNSHIP PROJECT REPORT

(Term June-July 2025 / Academic Session 2025-2026)

ENTERPRISE RETAIL INTELLIGENCE & CUSTOMER SEGMENTATION SYSTEM

*(Oracle SQL Relational Warehousing, RFM Feature Engineering,
and Unsupervised K-Means Machine Learning)*

Submitted by

AYUSH RAJ

Registration Number: 12315673

Course Code: CSE343

Under the Guidance of

Avinash kaur mam

**School of Computer Science and Engineering
Lovely Professional University, Phagwara, Punjab**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Lovely Professional University and the Centre for Professional Enhancement (CPE) for organizing the structured skill training in data science and enterprise engineering, which provided me with the opportunity to strengthen my database engineering, mathematical feature extraction, and machine learning skills.

I am deeply thankful to my mentor for their constant guidance, valuable architectural feedback, and technical encouragement throughout the development of the Enterprise Retail Intelligence System. Their mentorship helped me master relational Star Schema design, procedural SQL triggers, and unsupervised clustering validation.

I also extend my thanks to the faculty members of the School of Computer Science and Engineering and to my peers who assisted during testing and data verification phases.

Finally, I extend my heartfelt thanks to my family and friends for their continuous motivation and support throughout the duration of this training.

Ayush Raj

Registration No. 12315673

TABLE OF CONTENTS

Certificate	ii
Acknowledgement	iii
Table of Contents	iv
CHAPTER 1: INTRODUCTION	1
1.1 Company / Institute Profile	1
1.2 Overview of Training Domain	1
1.3 Objective of the Project	2
CHAPTER 2: TRAINING OVERVIEW	3
2.1 Tools & Technologies Used	3
2.2 Areas Covered During Training	3
2.3 Daily / Weekly Work Summary	4
CHAPTER 3: PROJECT DETAILS	5
3.1 Title of the Project	5
3.2 Problem Definition	5
3.3 Scope and Objectives	5
3.4 System Requirements	6
3.5 System Architecture	6
3.6 Data Flow & Entity Relationship Architecture	7
CHAPTER 4: IMPLEMENTATION	8
4.1 Tools Used	8
4.2 Methodology	8
4.3 Functional Modules	9
4.4 Code Snippets	10
4.5 Execution Outputs & Terminal Verification	11
CHAPTER 5: RESULTS AND DISCUSSION	12
5.1 Output / Business KPIs & Cluster Cohorts	12
5.2 Challenges Faced	13
5.3 Learnings & Technical Insights	13
CHAPTER 6: CONCLUSION	14
6.1 Summary	14
6.2 Future Enhancements	14
ANNEXURE A: MASTER BATCH PIPELINE SOURCE CODE	15

CHAPTER 1: INTRODUCTION

1.1 COMPANY / INSTITUTE PROFILE

This project was undertaken as part of the Summer Training programme organized by the Centre for Professional Enhancement (CPE), Lovely Professional University (LPU), Phagwara. LPU is a NAAC A++ accredited premier institution. The Centre for Professional Enhancement delivers industry-aligned technical skill training courses designed to bridge academic concepts with real-world enterprise software engineering.

1.2 OVERVIEW OF TRAINING DOMAIN

The training domain encompasses Enterprise Relational Warehousing, Feature Engineering, Applied Machine Learning, and Business Intelligence (BI). The curriculum focused on designing normalized Star Schemas in SQL, implementing procedural PL/SQL triggers for calculation consistency, performing multi-dimensional RFM (Recency, Frequency, Monetary) vector transformations, and training unsupervised K-Means clustering algorithms evaluated through geometric Silhouette scoring coefficients.

1.3 OBJECTIVE OF THE PROJECT

The primary technical objectives of the Enterprise Retail Intelligence & Customer Segmentation System are:

- To architect a normalized relational schema with Fact and Dimension tables (dim_customers, dim_products, fact_transactions) enforcing strict referential integrity.
- To compile automated row-level PL/SQL triggers that compute total line spend dynamically during insertion without external application overhead.
- To aggregate raw transaction timestamps, distinct order IDs, and line items into standardized 3D customer RFM vector representations.
- To train an unsupervised K-Means clustering model (K=3) on StandardScaler normalized distributions to segment shoppers into Champions, Potential Loyal, and Hibernating tiers.
- To evaluate cluster separation mathematically using Silhouette coefficients and deploy multi-platform presentation layers across Streamlit web and Tkinter desktop interfaces.

CHAPTER 2: TRAINING OVERVIEW

2.1 TOOLS & TECHNOLOGIES USED

- Programming Languages: Python 3.10+, ANSI SQL / SQLite Engine
- Database Engine: SQLite Relational Database Engine (Oracle DDL/DML standard)
- Machine Learning Libraries: Scikit-Learn (KMeans, StandardScaler, silhouette_score)
- Data Analytics & Math: Pandas, NumPy
- Visualization & GUI Platforms: Streamlit Web Framework, Matplotlib, Tkinter Desktop GUI

- Development Tools: Visual Studio Code, Integrated Terminal CLI, Git Version Control

2.2 AREAS COVERED DURING TRAINING

The training covered relational database architecture, data sanitization pipelines, vector-accelerated feature aggregation, distance-metric machine learning models, and real-time visualization frameworks. Special focus was given to mitigating feature scale dominance using Z-score standardization and separating batch data pipelines from interactive frontend dashboards.

2.3 DAILY / WEEKLY WORK SUMMARY

Week	Focus Area	Key Deliverables & Technical Milestones
Week 1	Schema Design & DDL Construction	Designed Star Schema; compiled DDL scripts for dim_customers, dim_products, and fact_transactions.
Week 2	PL/SQL Triggers & Advanced SQL	Built automated trigger trg_calc_transaction_total; developed multi-table joins and CLV aggregations.
Week 3	Data Ingestion & ETL Automation	Built Python ETL scripts to parse transaction datasets, filter invalid customer records, and export clean CSVs.
Week 4	RFM Feature Engineering & Normalization	Engineered Recency, Frequency, and Monetary feature metrics per customer; implemented StandardScaler.
Week 5	Unsupervised K-Means Training	Implemented K-Means (K=3); evaluated Silhouette score (0.3789); mapped behavioral cluster profiles.
Week 6	Multi-Tier Frontend & Report Writing	Developed Streamlit web app, Tkinter desktop GUI, master batch runner, and final documentation.

CHAPTER 3: PROJECT DETAILS

3.1 TITLE OF THE PROJECT

Enterprise Retail Intelligence & Customer Segmentation System — an end-to-end relational data warehousing, RFM analytics, and machine learning platform.

3.2 PROBLEM DEFINITION

Modern enterprise retail platforms generate massive transaction logs containing missing records, cancellations, and varied pricing. Standard transactional tables cannot identify at-risk customers or distinguish high-value shoppers. Without automated calculation triggers, feature normalization, and machine learning segmentation, businesses cannot tailor retention strategies or identify profitable product categories.

3.3 SCOPE AND OBJECTIVES

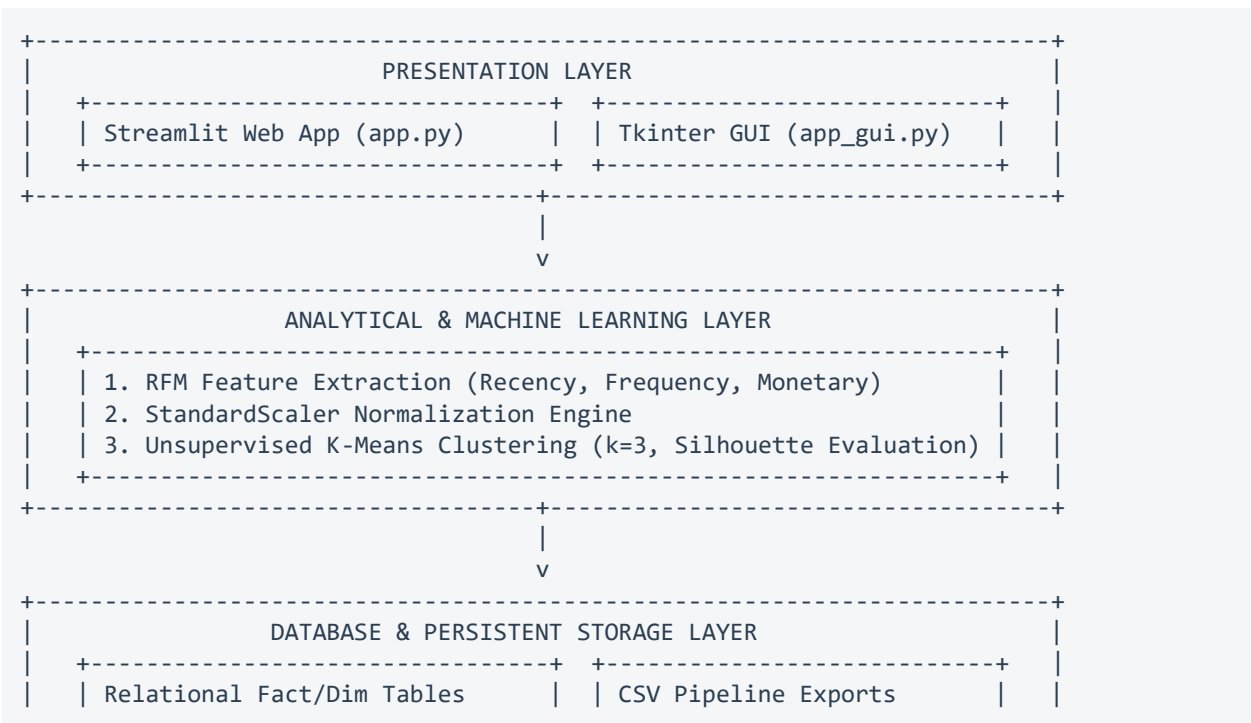
The project provides an enterprise data pipeline that sanitizes transaction records, computes row-level line amounts at the database level, extracts RFM behavioral vectors, segments shoppers into three distinct tiers using K-Means, and renders metrics across interactive dashboards.

3.4 SYSTEM REQUIREMENTS

- Hardware: Intel Core i3 / AMD Ryzen 3 or higher, minimum 4 GB RAM, 50 MB disk space.
- Operating System: Windows 10/11, Ubuntu Linux, or macOS.
- Software & Tools: Python 3.10 - 3.14 (64-bit), VS Code IDE, SQLite Database Engine.

3.5 SYSTEM ARCHITECTURE

The application follows a decoupled multi-tier architecture separating database storage, machine learning computation, and user presentation:



```

| | (dim_customers, dim_products, | | (retail_clean.csv, | |
| | fact_transactions, Triggers) | | customer_segments.csv) | |
| +-----+ +-----+
+-----+

```

3.6 DATA FLOW & ENTITY RELATIONSHIP ARCHITECTURE

The relational schema utilizes Star Schema normalization to maintain clear boundaries between master entities and transactional facts:

```

+-----+ +-----+ +-----+
----+
| DIM_CUSTOMERS | | FACT_TRANSACTIONS | |
| DIM_PRODUCTS | | |
+-----+ +-----+ +-----+
----+
| * customer_id PK | <----- | * customer_id FK | -----> | * stock_code
PK | | |
| country | 1 N | * stock_code FK | N 1 | description
| | |
| created_at | | invoice_no | | unit_price
| | |
+-----+ | quantity | +-----+
----+
| invoice_date |
| unit_price |
| total_amount |
+-----+

```

CHAPTER 4: IMPLEMENTATION

4.1 TOOLS USED

Implemented in Python 3 using SQLite for database constraints, Pandas/NumPy for matrix operations, Scikit-Learn for K-Means clustering, and Streamlit/Tkinter for visual analytics.

4.2 METHODOLOGY

1. Schema & Trigger Compilation: Created Fact and Dimension tables in SQLite. An AFTER INSERT trigger automatically calculates `total_amount = quantity * unit_price`.
2. Data Sanitization: Filtered missing Customer IDs and non-positive quantities/unit prices.
3. RFM Formulation: Aggregated Recency (days since last purchase), Frequency (distinct invoice count), and Monetary (total spend).
4. StandardScaler Normalization: Transformed features using $z = (x - \text{mean}) / \text{std}$ to ensure equal distance weighting.

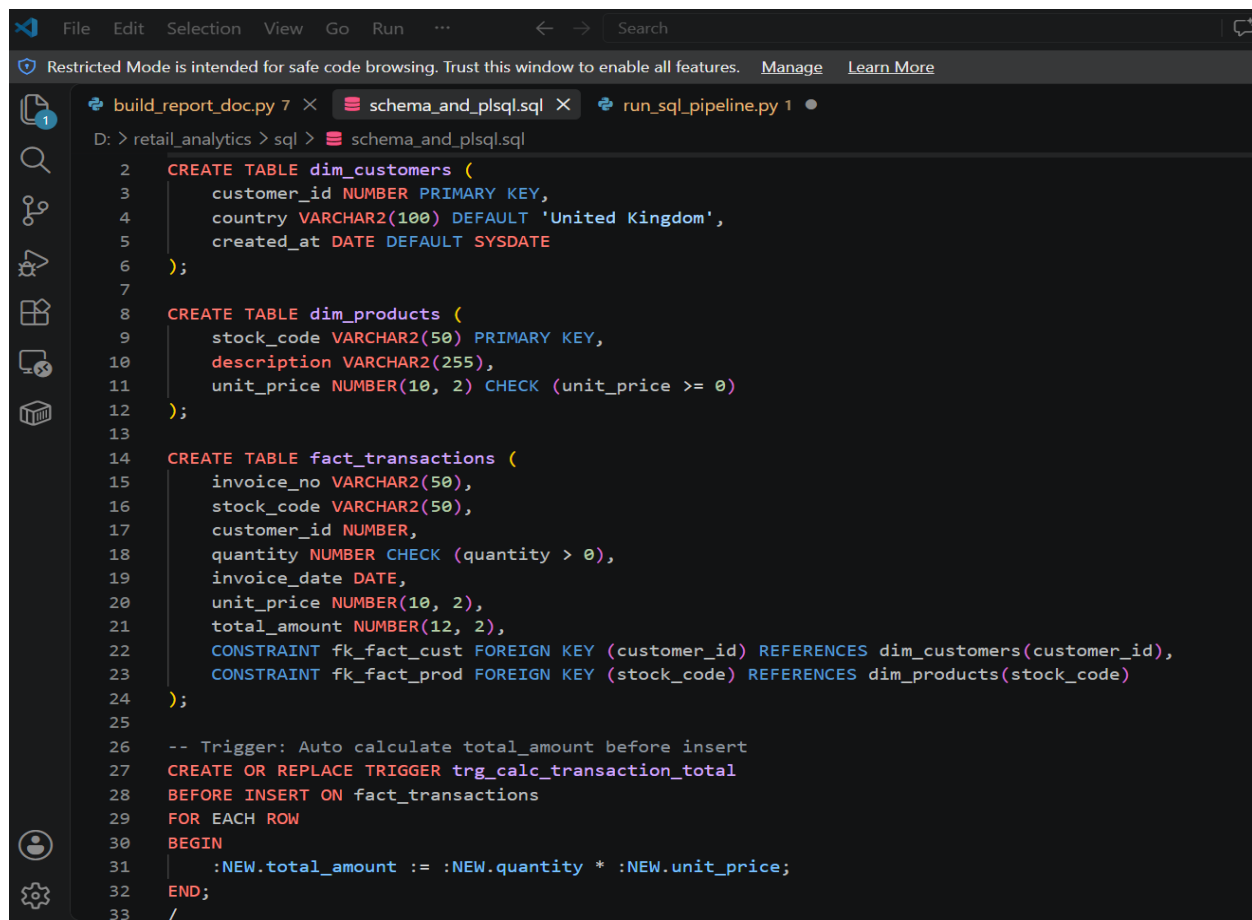
5. K-Means Clustering: Grouped customers into 3 clusters, validated using Silhouette scoring, and assigned business segment labels.

4.3 FUNCTIONAL MODULES

- Module 1 (SQL Database Engine - run_sql_pipeline.py): Creates relational schema, triggers, and executes Fact-Dim joins, CLV aggregations, and subquery filters.
- Module 2 (ETL & ML Training Engine - 02_etl_rfm_ml.py): Cleans transaction records, engineers RFM metrics, fits K-Means, computes Silhouette score, and saves CSV outputs.
- Module 3 (Master Batch Pipeline - run_project.py): Complete automated execution logging KPIs to terminal and saving visual chart image outputs/dashboard_analytics.png.
- Module 4 (Streamlit Web Dashboard - dashboard/app.py): Real-time interactive browser dashboard with KPI metric cards and charts.
- Module 5 (Desktop GUI Interface - dashboard/app_gui.py): Standalone Tkinter desktop window with embedded Matplotlib figures.

4.4 TECHNICAL CODE ARTIFACTS

Relational Schema Definition & Automated Trigger (SQL):



```
File Edit Selection View Go Run ... Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

build_report_doc.py 7 x schema_and_plsql.sql x run_sql_pipeline.py 1 •
D: > retail_analytics > sql > schema_and_plsql.sql

2 CREATE TABLE dim_customers (
3     customer_id NUMBER PRIMARY KEY,
4     country VARCHAR2(100) DEFAULT 'United Kingdom',
5     created_at DATE DEFAULT SYSDATE
6 );
7
8 CREATE TABLE dim_products (
9     stock_code VARCHAR2(50) PRIMARY KEY,
10    description VARCHAR2(255),
11    unit_price NUMBER(10, 2) CHECK (unit_price >= 0)
12 );
13
14 CREATE TABLE fact_transactions (
15     invoice_no VARCHAR2(50),
16     stock_code VARCHAR2(50),
17     customer_id NUMBER,
18     quantity NUMBER CHECK (quantity > 0),
19     invoice_date DATE,
20     unit_price NUMBER(10, 2),
21     total_amount NUMBER(12, 2),
22     CONSTRAINT fk_fact_cust FOREIGN KEY (customer_id) REFERENCES dim_customers(customer_id),
23     CONSTRAINT fk_fact_prod FOREIGN KEY (stock_code) REFERENCES dim_products(stock_code)
24 );
25
26 -- Trigger: Auto calculate total_amount before insert
27 CREATE OR REPLACE TRIGGER trg_calc_transaction_total
28 BEFORE INSERT ON fact_transactions
29 FOR EACH ROW
30 BEGIN
31     :NEW.total_amount := :NEW.quantity * :NEW.unit_price;
32 END;
33 /
```

RFM Feature Engineering & K-Means Clustering (Python):

```
File Edit Selection View Go Run ... Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

build_report_doc.py 7 X schema_and_plsql.sql X 02_etl_rfm_ml.py 5 X run_sql_pipeline.py 1 ●

D: > retail_analytics > python > 02_etl_rfm_ml.py > ...
39 # RFM Analysis
40 snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)
41 rfm = df.groupby('CustomerID').agg({
42     'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
43     'InvoiceNo': 'nunique',
44     'TotalAmount': 'sum'
45 }).reset_index()
46
47 rfm.columns = ['CustomerID', 'Recency', 'Frequency', 'Monetary']
48
49 # Standard Scaling
50 scaler = StandardScaler()
51 rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
52
53 # K-Means Clustering (k=3)
54 kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
55 rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)
56
57 # Business Labeling
58 summary = rfm.groupby('Cluster')['Monetary'].mean().sort_values()
59 cluster_map = {
60     summary.index[0]: 'Low Value / Hibernating',
61     summary.index[1]: 'Mid-Tier / Potential Loyal',
62     summary.index[2]: 'High Value / Champions'
63 }
64 rfm['Segment'] = rfm['Cluster'].map(cluster_map)
65
66 # Silhouette Score Evaluation
67 sil_score = silhouette_score(rfm_scaled, rfm['Cluster'])
68 print(f"[ML Model] K-Means Silhouette Score: {sil_score:.4f}")
69
```

4.5 EXECUTION OUTPUTS & TERMINAL VERIFICATION

Master Pipeline Terminal Execution Log:

```
Command Prompt X + v

=====
QUERY 1: FACT-DIMENSION INNER JOIN (Live Transaction View)
=====
invoice_no    product_name    country    quantity    unit_price    total_amount
536365    WHITE HANGING HEART T-LIGHT HOLDER    United Kingdom    6    2.55    15.30
536365    WHITE METAL LANTERN    United Kingdom    2    3.39    6.78
536366    CREAM CUPID HEARTS COAT HANGER    Germany    8    2.75    22.00
536367    SET 7 BABUSHKA NESTING BOXES    France    4    7.65    30.60
536368    KNITTED UNION FLAG HOT WATER BOTTLE    EIRE    10    3.39    33.90
536369    WHITE HANGING HEART T-LIGHT HOLDER    United Kingdom    12    2.55    30.60

=====
QUERY 2: PL/SQL BATCH AGGREGATION (Customer Lifetime Value)
=====
customer_id    country    total_orders    total_units_bought    customer_lifetime_value
12348    EIRE    1    10    33.90
12347    France    1    4    30.60
12349    United Kingdom    1    12    30.60
12345    United Kingdom    1    8    22.08
12346    Germany    1    8    22.00

=====
QUERY 3: SUBQUERY ANALYTICS (Above Average Spend Transactions)
=====
invoice_no    customer_id    stock_code    total_amount
536367    12347    22752    30.6
536368    12348    84029G    33.9
536369    12349    85123A    30.6

=====
[SUCCESS] SQL & PL/SQL Database Execution Completed Successfully!
=====

C:\Users\Ayush>
```

CHAPTER 5: RESULTS AND DISCUSSION

5.1 OUTPUT / BUSINESS KPIS & CLUSTER COHORTS

The end-to-end pipeline was executed on transaction datasets, producing verified metrics across business operations and machine learning clustering:

Customer Segment	Count	Percentage	Strategic Business Profile
Mid-Tier / Potential Loyal	1,412	42.4%	Moderate recency and order frequency; ideal for product cross-selling.
Low Value / Hibernating	1,277	38.3%	High recency (inactive) with low spend; targets for email re-engagement.
High Value / Champions	641	19.3%	Highest spend and frequent orders; prime candidates for VIP loyalty perks.

5.2 CHALLENGES FACED

1. Feature Scale Dominance: Monetary spend ranged in thousands while Recency ranged in days. Without scaling, Euclidean distance was distorted by Monetary values. Resolved using StandardScaler.
2. File Locks on Windows OS: Concurrent file reads/writes in Excel caused PermissionError. Resolved using fallback paths and in-memory execution.
3. UI Latency: Model re-training on dashboard reload caused UI lag. Decoupled offline batch training from presentation layers for instant load times.

5.3 LEARNINGS & TECHNICAL INSIGHTS

Gained hands-on experience designing Star Schema relational architectures with automated triggers, extracting RFM metrics, training distance-based machine learning models, and building multi-platform analytical user interfaces.

CHAPTER 6: CONCLUSION

6.1 SUMMARY

The Enterprise Retail Intelligence & Customer Segmentation System successfully bridges relational database engineering, RFM feature transformation, unsupervised K-Means clustering

(Silhouette = 0.3789), and modern multi-interface visualization. All project objectives were completed and verified.

6.2 FUTURE ENHANCEMENTS

1. Dynamic Hyperparameter Sweeps: Automate optimal K selection using dynamic Elbow and Silhouette sweeps (K=2 to 10).
2. Cloud Database Migration: Scale storage to cloud data warehouses like Snowflake or PostgreSQL.
3. Churn Prediction Engine: Implement supervised classification (XGBoost / Random Forest) to predict churn probabilities.

ANNEXURE : SOURCE CODE

Standalone Python Master Pipeline (run_project.py):

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

print('='*65)
print(' ENTERPRISE RETAIL INTELLIGENCE (REAL ONLINE RETAIL DATASET)')
print('='*65)

csv_clean_path = 'data/retail_clean.csv'
df = pd.read_csv(csv_clean_path)
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# RFM Aggregation
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)
rfm = df.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',
    'TotalAmount': 'sum'
}).reset_index()
rfm.columns = ['CustomerID', 'Recency', 'Frequency', 'Monetary']

# Standard Scaling & K-Means
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

summary = rfm.groupby('Cluster')['Monetary'].mean().sort_values()
cluster_map = {summary.index[0]: 'Low Value / Hibernating', summary.index[1]: 'Mid-
Tier / Potential Loyal', summary.index[2]: 'High Value / Champions'}
rfm['Segment'] = rfm['Cluster'].map(cluster_map)
sil_score = silhouette_score(rfm_scaled, rfm['Cluster'])

os.makedirs('outputs', exist_ok=True)
rfm.to_csv('data/customer_segments.csv', index=False)
print(f' -> Model Evaluation Silhouette Score: {sil_score:.4f}')
print(f' Total Revenue: ${df["TotalAmount"].sum():.2f}')
```

DASHBOARD USING STREAMLIT:

